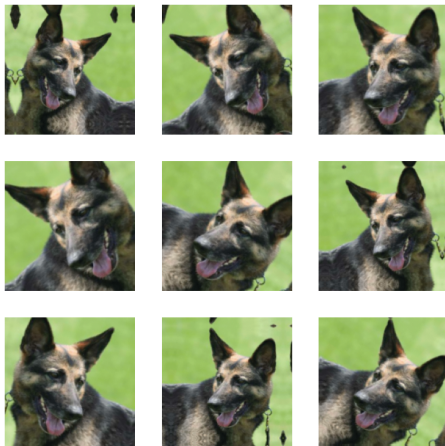


Kỹ thuật Tăng cường Dữ liệu (Data Augmentation)

- Không có thêm dữ liệu? Hãy **tự tạo ra dữ liệu mới** từ dữ liệu cũ!
- Áp dụng các phép biến đổi ngẫu nhiên: Xoay (Rotation), Dịch chuyển (Translation), Thu phóng (Zoom), Lật ngang (Horizontal Flip).
- Mô hình sẽ không bao giờ nhìn thấy cùng một bức ảnh 2 lần, buộc nó phải học các đặc trưng *bất biến*.



Hình 8.11: Cùng một con mèo nhưng sinh ra nhiều biến thể

Data Augmentation: Mã nguồn

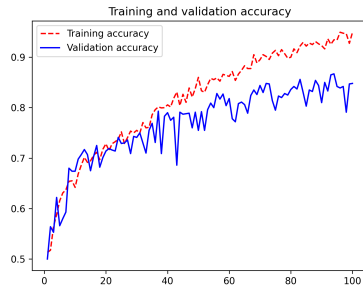
Keras cung cấp sẵn các Layer tăng cường dữ liệu. Các lớp này có thể được gắn thẳng vào mô hình.

```
1 data_augmentation = keras.Sequential(  
2     [  
3         layers.RandomFlip("horizontal"),  
4         layers.RandomRotation(0.1),  
5         layers.RandomZoom(0.2),  
6     ]  
7 )  
8  
9 # Insert augmentation layer before the main ConvNets  
10 inputs = keras.Input(shape=(180, 180, 3))  
11 x = data_augmentation(inputs)  
12 x = layers.Rescaling(1./255)(x)  
13 x = layers.Conv2D(filters=32, kernel_size=3, activation="relu")(x)  
14 # ... Add more Convolutional Blocks ...
```

Lưu ý: Data Augmentation tự động bị Vô hiệu hóa (Deactivated) trong quá trình Testing / Validation.

Đánh giá Accuracy sau Augmentation

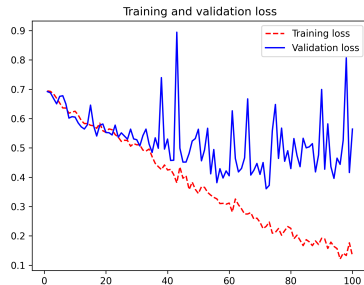
- Validation Accuracy đã tăng từ 70% lên **hơn 80%**.
- Quan trọng hơn: Đường Validation (Đứt nét) bám sát đường Training (Xanh).
- Mô hình hoàn toàn không còn hiện tượng Overfitting, và chúng ta có thể tiếp tục huấn luyện lâu hơn.



Hình 8.12: Accuracy có Data Augmentation

Đánh giá Loss sau Augmentation

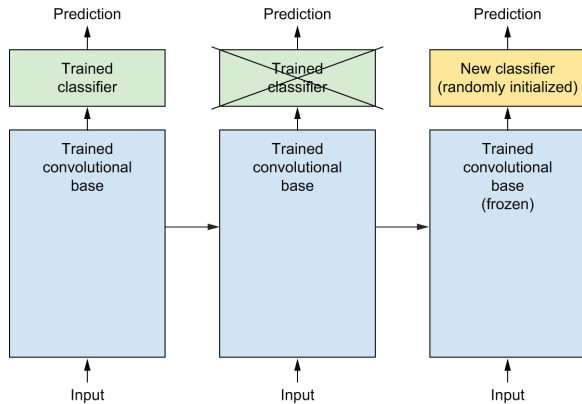
- Đồ thị Loss không còn bị nảy ngược lên như trước.
- Kỹ thuật Augmentation được tính toán song song trên CPU/GPU qua hàm `tf.data.pipeline` mà không làm chậm quá trình Train.
- **Giới hạn:** Ta vẫn đang bị kẹt ở mức 82%. Để vượt ngưỡng này, ta cần sức mạnh từ Transfer Learning.



Hình 8.13: Loss có Data Augmentation

Trích xuất Đặc trưng (Feature Extraction)

- **Khái niệm:** Tái sử dụng một mô hình cực lớn (VGG16) đã được huấn luyện trên 1.4 triệu bức ảnh của ImageNet.
- **Kiến trúc:** Giữ nguyên Khối Tích chập (Convolutional Base) của VGG16.
- Cắt bỏ phần đầu Phân loại (Fully Connected Classifier) cũ của VGG16, và **Gắn lõi Phân loại mới** của chúng ta vào.



Hình 8.14: Thay thế bộ phân loại (Swapping FC)

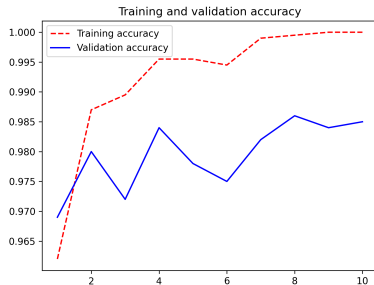
Feature Extraction: Mã nguồn (Sử dụng VGG16)

Khởi tạo mô hình VGG16, sau đó "Đóng băng" (Freeze) toàn bộ trọng số của Base.

```
1 # Load pre-trained model without the top dense layers (include_top=False)
2 conv_base = keras.applications.vgg16.VGG16(
3     weights="imagenet",
4     include_top=False,
5     input_shape=(180, 180, 3)
6 )
7
8 # Freeze the base!
9 conv_base.trainable = False
10
11 # Attach our custom classifier
12 inputs = keras.Input(shape=(180, 180, 3))
13 x = conv_base(inputs)
14 x = layers.Flatten()(x)
15 x = layers.Dense(256, activation="relu")(x)
16 x = layers.Dropout(0.5)(x)
17 outputs = layers.Dense(1, activation="sigmoid")(x)
18 model = keras.Model(inputs, outputs)
```

Accuracy với Feature Extraction Cơ bản

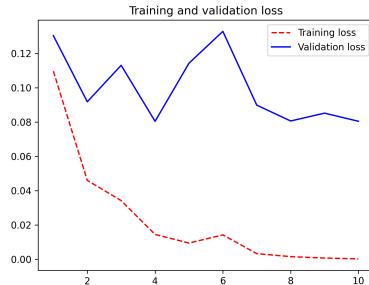
- Bằng cách đóng băng (Freeze) VGG16 Base, ta chỉ huấn luyện lớp Dense mới thêm vào.
- Kết quả đáng kinh ngạc: Validation Accuracy vọt lên **90%**.
- Sự mạnh mẽ của ImageNet đã giúp trích xuất các đặc trưng mắt mèo/mũi chó cực kỳ tinh xảo.



Hình 8.15: Accuracy Feature Extraction

Loss với Feature Extraction Cơ bản

- Tuy nhiên, vì tốc độ học quá nhanh, đồ thị Loss lại tiếp tục báo hiệu **Overfitting cực kỳ nặng** ngay từ Epoch thứ 5.
- Tại sao? Vì ta chưa áp dụng Data Augmentation cho quy trình này.



Hình 8.16: Loss Feature Extraction (Bị Overfitting)

Mô hình Liên hoàn: Feature Extraction + Data Augmentation

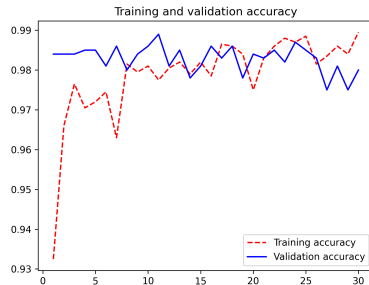
Để giải quyết bài toán Overfitting, ta kết hợp Data Augmentation vào chuỗi tính toán.

```
1 inputs = keras.Input(shape=(180, 180, 3))
2 # 1. Apply Augmentation
3 x = data_augmentation(inputs)
4 # 2. Preprocess specifically for VGG16
5 x = keras.applications.vgg16.preprocess_input(x)
6 # 3. Extract features via Frozen Base
7 x = conv_base(x)
8 # 4. Custom classifier
9 x = layers.Flatten()(x)
10 x = layers.Dense(256)(x)
11 x = layers.Dropout(0.5)(x)
12 outputs = layers.Dense(1, activation="sigmoid")(x)
13 model = keras.Model(inputs, outputs)
```

Đây là kỹ thuật yêu cầu GPU mạnh vì phải chạy forward-pass qua toàn bộ Base nhiều lần.

Accuracy với Feature Extraction + Data Augmentation

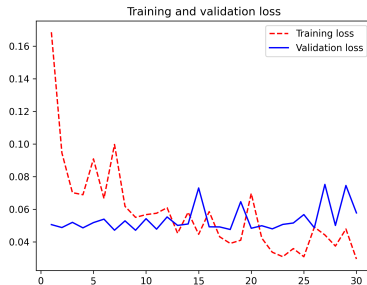
- Ta thiết lập một kiến trúc liên hoàn: Lớp Augmentation → VGG16 Base (Đóng băng) → Lớp Dense (Huấn luyện).
- Kết quả: **Mượt mà hoàn hảo**. Validation bám sát Training và không còn Overfitting. Đạt mốc **96%**.



Hình 8.17: Accuracy FE + Data Augmentation

Loss với Feature Extraction + Data Augmentation

- Đồ thị Loss cực kỳ đẹp.
- Quá trình này đòi hỏi máy phải có card đồ họa (GPU), vì VGG16 phải chạy dự đoán lại từ đầu qua mỗi bức ảnh biến đổi ngẫu nhiên.



Hình 8.18: Loss FE + Data Augmentation

Để ép hiệu suất mô hình đến giới hạn cực đại:

- ➊ Gắn bộ phân loại tùy chỉnh lên trên một mô hình cơ sở (VGG16).
- ➋ Đóng băng toàn bộ mô hình cơ sở (Freeze Base).
- ➌ Huấn luyện bộ phân loại tùy chỉnh.
- ➍ **Mở khóa (Unfreeze)** một vài lớp Tích chập trên cùng của Base.
- ➎ Huấn luyện đồng thời cả bộ phân loại và các lớp Tích chập vừa mở khóa với **Tốc độ học cực nhỏ (Rất nhỏ)**.

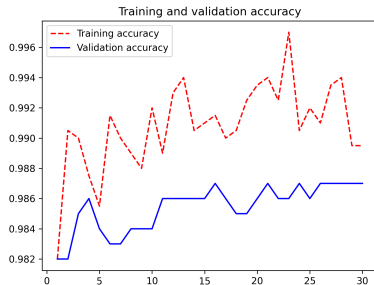
Fine-Tuning: Mã nguồn giải mã các lớp cuối

Chỉ mở khóa Block cuối cùng của VGG16 (block 5).

```
1 conv_base.trainable = True
2
3 # Freeze all layers EXCEPT the ones from 'block5_conv1' onwards
4 set_trainable = False
5 for layer in conv_base.layers:
6     if layer.name == "block5_conv1":
7         set_trainable = True
8     if set_trainable:
9         layer.trainable = True
10    else:
11        layer.trainable = False
12
13 # Re-compile model with a VERY SMALL learning rate
14 model.compile(
15     loss="binary_crossentropy",
16     optimizer=keras.optimizers.RMSprop(learning_rate=1e-5),
17     metrics=["accuracy"]
18 )
```

Kết quả Accuracy sau Fine-Tuning

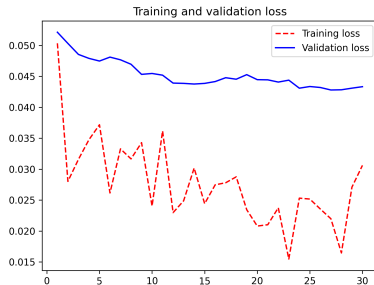
- Bằng cách nới lỏng nhẹ trọng số của khối Block 5 (VGG16), mô hình tự điều chỉnh các đặc trưng bậc cao sao cho khớp hoàn hảo với chó/mèo.
- Độ chính xác vọt lên **97.5%**. Đánh bại hoàn toàn mô hình cơ sở ban đầu.



Hình 8.19: Accuracy sau Fine-Tuning

Kết quả Loss sau Fine-Tuning

- Đồ thị Loss có thể biến động nhẹ (nhiều), nhưng giá trị trung bình trượt dài (Moving Average) cho thấy sự ổn định.
- Mặc dù chỉ dùng 2000 ảnh (so với hàng chục ngàn ảnh của người thắng giải Kaggle), ta vẫn đạt kết quả xuất chúng nhờ Transfer Learning.



Hình 8.20: Loss sau Fine-Tuning